



Swinburne University Of Technology
School of Science, Computing, and Engineering Technologies
ASSIGNMENT AND PROJECT COVER SHEET

Unit Code: COS30015 Unit Title: IT Security

Assignment number and title: Assignment 1 Due date: _____

Lab Group: _____ Tutor: _____ Lecturer: Hoang Xuan Tung

Family name: Nguyen Dinh Dung Identity no: 104772138

Other names: _____

To be completed if this is an INDIVIDUAL ASSIGNMENT

I declare that this assignment is my individual work. I have not worked collaboratively nor have I copied from any other student's work or from any other source except where due acknowledgment is made explicitly in the text, nor has any part been written for me by another person.

Signature: Dung

To be completed if this is a GROUP ASSIGNMENT

We declare that this is a group assignment and that no part of this submission has been copied from any other student's work or from any other source except where due acknowledgment is made explicitly in the text, nor has any part been written for us by another person.

ID Number	Name	Signature
_____	_____	_____
_____	_____	_____

Marker's comments:

Total Mark: _____

Extension certification:

This assignment has been given an extension and is now due on _____

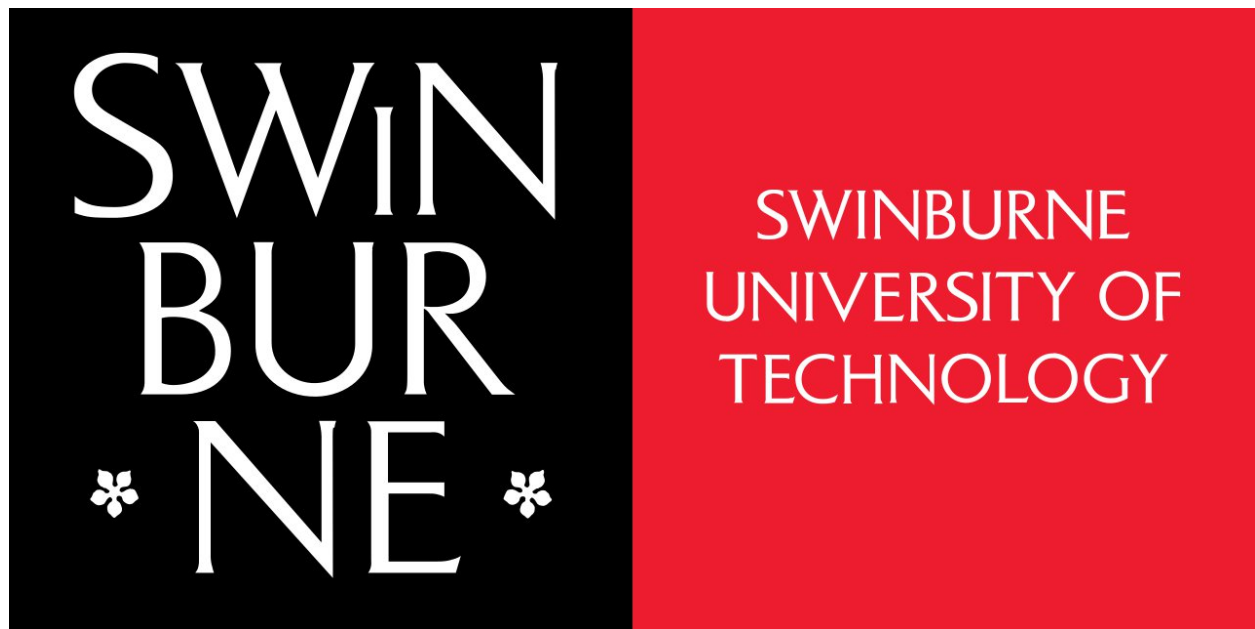
Signature of Convener: _____ Date: /2025

COS30015 – IT Security

Assignment 1

Student Name: Nguyen Dinh Dung

Student ID: 104772138



Topic: Denial-of-Service

Introduction

The preservation of the availability pillar within the Confidentiality, Integrity, and Availability (CIA) triad remains one of the most significant challenges in modern information security [1]. Denial of Service (DoS) attacks represent a fundamental threat to this pillar, having evolved significantly from simple bandwidth-exhaustion techniques to sophisticated, low-bandwidth application-layer maneuvers [2]. Among these, the Slowloris attack stands as a definitive example of a "low-and-slow" threat, targeting the connection management mechanisms of thread-based web servers rather than attempting to saturate network pipes [3]. Denial of Service is defined as a malicious attempt to render a computer, device, or network infrastructure unavailable to its intended users. While volumetric attacks like ICMP or UDP floods rely on overwhelming bandwidth, and protocol attacks like SYN floods exploit the Transmission Control Protocol (TCP) three-way handshake, application-layer (Layer 7) attacks focus on specific protocol vulnerabilities—most notably those found in Hypertext Transfer Protocol (HTTP). Slowloris, originally conceptualized by Robert "RSnake" Hansen in 2009, operates by opening numerous connections to a target web server and maintaining them indefinitely through the periodic transmission of partial HTTP headers [4]. The emergence of such techniques has forced a reassessment of traditional defensive postures. Because Slowloris sends legitimate-looking HTTP requests, standard network-layer defenses like firewalls often fail to distinguish malicious connections from valid client traffic. This exhaustive research report investigates the Slowloris attack methodology through a comprehensive case study, comparative tool analysis, and practical testing within a controlled virtual environment. It provides a detailed justification for the selection of specific offensive and defensive tools, followed by an in-depth analysis of the technical mechanisms that facilitate service exhaustion. By mapping these activities to the MITRE ATT&CK framework and evaluating them against the ACSC Essential Eight, this analysis bridges the gap between theoretical offensive techniques and applied defensive excellence.

Theoretical Framework of Availability and Denial of Service

Availability ensures that information and resources are accessible to authorized users when needed. In the context of a web server, this requires the continuous functioning of the network interface, the operating system's process management, the web server software (such as Apache or Nginx), and the backend application logic. A DoS attack seeks to break this chain by consuming a critical resource to the point of exhaustion [3]. The taxonomy of DoS attacks typically classifies threats based on the Open Systems Interconnection (OSI) layer they target. Network-layer attacks (Layers 3 and 4) like SYN floods and UDP floods attempt to saturate the bandwidth or the packet-processing capacity of network devices. In contrast, application-layer attacks (Layer 7) target specific services provided by the system, such as web servers or DNS [1]. Slowloris is unique because it combines a low-bandwidth footprint with a high-impact outcome, exploiting the semantic rules of HTTP rather than the volume of packets [3]. The risk posed by application-layer DoS is magnified by the shift toward digital business models. For contemporary organizations, the website is often the primary portal for revenue, customer service, and internal operations. Consequently, the ability of a single adversary with minimal resources to take down an enterprise-grade server represents a critical vulnerability in the digital ecosystem [3].

Case Study: Historical Analysis and Business Impact of Slow

HTTP Attacks

The history of the Slowloris attack methodology demonstrates its evolution from a niche security researcher's tool to a standard component of the hacktivist and state-actor arsenal [4]. By analyzing real-world deployments, the analysis reveals the strategic value adversaries place on "stealth and efficiency" over raw volume. One of the most notable early deployments of Slowloris occurred during the 2009 Iranian presidential election protests [4].

Year	Target/Incident	Attacker/Context	Primary Impact
2009	Iranian Gov Sites	Political Protesters	Disrupted state communications during civil unrest while preserving protester bandwidth [5].
2011	CIA Website	LulzSec Group	Website rendered unavailable for several hours; demonstrated the tool's potency against high-profile targets [6].
2024	Internet Archive	Unidentified Actor	A massive attack involving modern variations of slow DoS took the service offline for an extended period [2].

Table 1. Significant attack cases using Slowloris from 2009 until now

2009 – Iranian Government Sites

During the 2009 Iranian presidential election protests, activists used Slowloris to target government websites such as gerdab.ir and leader.ir. Instead of flooding the servers with high traffic, they opened many HTTP connections and sent incomplete request headers very slowly, keeping connections alive and exhausting the server's available threads. This tactic was strategically chosen because it consumed minimal bandwidth, allowing protesters to preserve their own internet access while disrupting state communications. The result was temporary outages of government websites.

2024 – Internet Archive Attack (Modern Slow/Hybrid DDoS)

In 2024, the Internet Archive suffered a large-scale, multi-wave DDoS attack that combined modern HTTP flood techniques with slow DoS variations. Attackers generated massive volumes of legitimate-looking HTTP requests while also using "low-and-slow" tactics, such as sending large declared content-length requests but transmitting the body extremely slowly to tie up server resources. Unlike the

2009 case, this attack involved distributed infrastructure and sustained waves, overwhelming the Archive’s systems and forcing extended service outages and temporary read-only modes.

Both incidents demonstrate the effectiveness of application-layer DoS attacks, particularly “low-and-slow” techniques that exhaust server resources without necessarily requiring massive bandwidth. The 2009 attack highlighted how Slowloris can be tactically efficient in politically sensitive situations, while the 2024 incident showed how modern attackers combine slow techniques with large-scale distributed floods for greater impact. Effective defense requires layered protection: connection management tuning, rate limiting, web application firewalls, automated traffic filtering, infrastructure scaling, and dedicated DDoS mitigation services to detect and neutralize abnormal traffic patterns in real time. These incidents illustrate a consistent trend: the shift toward "stealth". By mimicking legitimate traffic patterns, Slowloris and its variants allow attackers to bypass simple volumetric detection mechanisms, often remaining unnoticed until service failure occurs.

Business and Organizational Consequences

The impact of a successful Slowloris attack extends far beyond technical downtime. For e-commerce platforms, unavailability during peak periods, such as product launches or holiday sales, results in direct and immediate financial loss [3]. Service-based organizations face reputational damage that can take years to recover, as customers lose confidence in the reliability of their digital services. Furthermore, contemporary research indicates that DoS attacks are often used as "smokescreens". While the IT security team is occupied with mitigating a visible service outage, the adversary may be conducting more insidious activities, such as data exfiltration or credential stuffing, under the cover of the logging "noise" generated by the DoS event. This makes the detection and mitigation of Slowloris a critical component of a broader defense-in-depth strategy. From a resource perspective, Slowloris is devastatingly efficient. Unlike traditional DDoS, which might require a botnet of 100,000 infected devices to generate a Terabit-per-second flood, Slowloris can be executed from a single machine with a regular home internet connection [3]. This low barrier to entry democratizes the capability for high-impact disruption, making it a threat that organizations of all sizes must address.

Planning and Justification

Comparative Tool Analysis: Offensive Perspectives

Adversaries have several options for conducting application-layer DoS attacks. A comparison of Slowloris against the High Orbit Ion Cannon (HOIC) [7] and GoldenEye [8] highlights the unique characteristics that make Slowloris [9] a primary concern for thread-based servers.

Criteria	Slowloris	GoldenEye	HOIC (High Orbit Ion Cannon)
Primary Mechanism	Partial HTTP headers; keeps connections open.	Rapidly requests URLs; Keep-Alive/No-Cache abuse.	Multi-threaded HTTP flood with booster scripts.
Strategy	Low-and-Slow	Hybrid (Volume +	High-Volume

	(Semantic).	Logic).	(Volumetric).
Bandwidth Req.	Extremely Low.	Moderate.	High (requires bandwidth to flood).
Interface Type	CLI (Perl/Python/Go).	CLI (Python-based).	GUI (Windows standalone).
Stealth Level	High; bypasses many IDS.	Moderate; heavy load.	Low; easily identified as a flood.
Target Vulnerability	Thread-based server connection pools.	Web server and backend application resources.	Network pipe and CPU throughput.

Table 2. Evaluation criteria for the attackers' tools

Slowloris is chosen for this analysis because it represents the most sophisticated exploitation of server-side architectural logic. While HOIC relies on brute-force volume that can often be mitigated by upstream ISP scrubbing, Slowloris represents a configuration and architectural challenge that requires granular server-level tuning and monitoring. GoldenEye is frequently used for stress-testing and simulations but behaves more like a traditional high-request flood, whereas Slowloris focuses specifically on the "idle wait" state of the connection table.

Comparative Tool Analysis: Defensive Perspectives

Defending against Slowloris in an Apache environment involves selecting between various modules and Multi-Processing Module (MPM) configurations. The primary defensive focus in this analysis is the use of `mod_reqtimeout` and the transition from `mpm_prefork` to `mpm_event`.

Mitigation	Technical Mechanism	Justification for/against Use
mod_reqtimeout	Enforces temporal limits and minimum rates for headers/bodies.	Selected: Official Apache solution; native, low overhead, and highly effective against slow requests.
MPM Event	Listener thread offloads idle sockets; efficient connection management.	Selected: Dramatically improves connection capacity; standard for modern high-load Apache 2.4+.
mod_qos	Advanced QoS logic, including per-IP connection limits.	Alternative: Powerful but complex; risk of blocking legitimate users if misconfigured.
mod_evasive	Simple rate limiting based on request frequency.	Dismissed: Ineffective against Slowloris as the request rate is

		too low to trigger traditional thresholds.
--	--	--

Table 3. Evaluation criteria for the defenders’ tools

The combination of mod_reqtimeout and mpm_event is justified as the industry-standard baseline for DoS resilience [10]. While modules like mod_antiloris or mod_qos provide additional granularity (e.g., per-IP connection limits), mod_reqtimeout addresses the root cause of the Slowloris exploit: the server's willingness to wait indefinitely for incomplete data.

Testing Scenario and Metrics

The analysis is conducted within a host-only virtual networking (VMnet1) environment to prevent any impact on live systems, as mandated by the assignment requirements. The environment (Figure 1) consists of a Kali Linux (2025) attacker machine and an Red Hat Enterprise Linux (10.1) target running Apache 2.4.

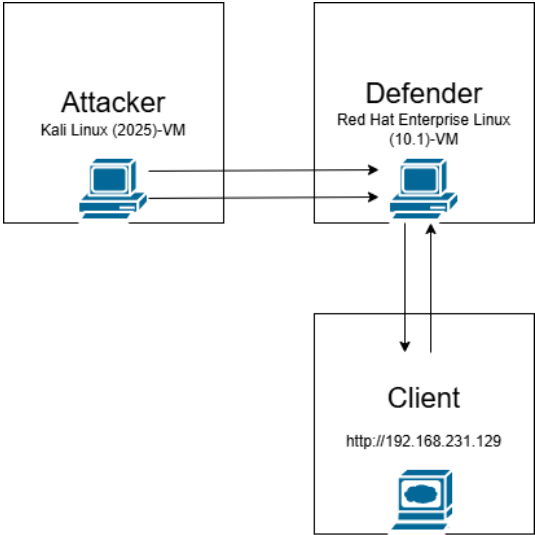


Figure 1. Environment setup

The experiment is divided into two phases:

- Phase 1 (Baseline):** The attacker launches a Slowloris attack against the Apache server using default configurations (mpm_prefork and no specific timeout modules active). The impact on service availability is observed and recorded.
- Phase 2 (Hardened):** The server is reconfigured to use mpm_event and mod_reqtimeout. The attack is repeated, and the server’s ability to maintain availability for legitimate clients is assessed under the same adversarial conditions.

Metrics for evaluating the success of the attacker versus the defender include:

Metric	Definition	Success Threshold (Win)
HTTP Availability	Successful retrieval of the server’s index page by a	100% Success (Defender Win) / 0% Success (Attacker Win)

	legitimate user during an attack.	
Response Latency	Time taken (in seconds) for a legitimate GET request to complete.	< 2s (Defender Win) / > 30s or Connection Reset (Attacker Win)
Connection Table State	The ratio of occupied worker threads to the total configured MaxRequestWorkers.	< 50% (Defender Win) / 100% (Attacker Win)
Logging Accuracy	The presence of indicative log messages (e.g., AH00161 vs. 408).	408 errors logged (Mitigation active) / AH00161 logged (Exhaustion reached).

Table 4. Metrics for evaluating the success of the attacker versus the defender

MITRE ATT&CK Mapping

Part 1: MITRE Tactics

Tactics represent the "why" of an ATT&CK technique—the adversary's technical goals and the reason for performing an action.

Tactic ID	Tactic Name	Relevance to Slowloris
TA0043	Reconnaissance	The attacker identifies target web server endpoints and assesses their susceptibility to slow attacks by identifying the server type and version (Apache/2.X).
TA0042	Resource Development	The adversary acquires the necessary tools (Slowloris script) and infrastructure to execute the attack.
TA0040	Impact	The primary tactic. The adversary seeks to disrupt or degrade the availability of systems and data, effectively rendering the web service unusable for legitimate traffic.

Table 5. Mapping test case to MITRE tactics

Part 2: MITRE Techniques

Techniques represent the "how" an adversary achieves a tactical goal by performing an action.

Technique ID	Technique Name	Justification for Mapping
T1499.002	Service Exhaustion Flood	Slowloris specifically targets the network services (HTTP/HTTPS) provided by the system, aiming to exhaust the connection pool without saturating the network bandwidth.
T1499.003	Application Exhaustion Flood	The adversary targets the application's connection handling logic, sending repetitive partial requests to exhaust system resources (threads/processes).
T1071	Application Layer Protocol	The attack leverages standard HTTP protocol interactions. By sending legitimate-looking headers, the attacker avoids detection by signatures that look for malformed packets.

Table 6. Mapping test case to MITRE techniques

Application and Documentation

Setting up

1. On Redhat

- Install Apache (Figure 2):

```
admin@localhost:~$ yum install httpd
```

Figure 2. Install Apache

- For better visualization, instead of using the default Apache welcome page, I will create an index.html (Figure 3) as the target webpage in this folder structure /var/www/html/index.html:

```
admin@localhost:~$ sudo nano /var/www/html/index.html
[sudo] password for admin:
admin@localhost:~$ sudo tee /var/www/html/index.html << 'EOF'
<h1>Target Web Server</h1>
EOF
```

Figure 3. index.html

- Then change the SELinux security context, set type to "httpd system content" (Figure 4):

```
admin@localhost:~$ sudo chcon -R -t httpd_sys_content_t /var/www/html
```

Figure 4. Command to change SELinux

- Try access the website (Figure 5):

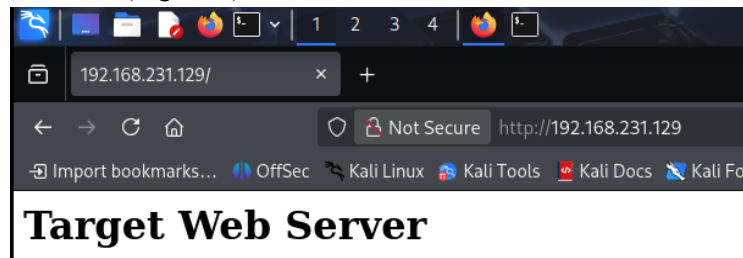


Figure 5. Target web server

- Disable mod_reqtimeout if enabled (Figure 6):

```
admin@localhost:~$ sudo sed -i 's/LoadModule reqtimeout_module/#LoadModule reqtimeout_module/' /etc/httpd/conf.modules.d/*
```

Figure 6. Disable mod_reqtimeout

- Switch to prefork MPM if on event MPM (Figure 7, 8):

```
admin@localhost:~$ sudo nano /etc/httpd/conf.modules.d/00-mpm.conf  
[sudo] password for admin:
```

Figure 7. edit conf file

```
GNU nano 8.1 /etc/httpd/conf.modules.d/00-mpm.conf Modified  
# prefork MPM: Implements a non-threaded, pre-forking web server  
# See: http://httpd.apache.org/docs/2.4/mod/prefork.html  
#  
# NOTE: If enabling prefork, the httpd_graceful_shutdown SELinux  
# boolean should be enabled, to allow graceful stop/shutdown.  
#  
LoadModule mpm_prefork_module modules/mod_mpm_prefork.so  
  
# worker MPM: Multi-Processing Module implementing a hybrid  
# multi-threaded multi-process web server  
# See: http://httpd.apache.org/docs/2.4/mod/worker.html  
#  
#LoadModule mpm_worker_module modules/mod_mpm_worker.so  
  
# event MPM: A variant of the worker MPM with the goal of consuming  
# threads only for connections with active processing  
# See: http://httpd.apache.org/docs/2.4/mod/event.html  
#  
#LoadModule mpm_event_module modules/mod_mpm_event.so
```

Figure 8. Line added

2. On Kali

- Install Slowloris (Figure 9):

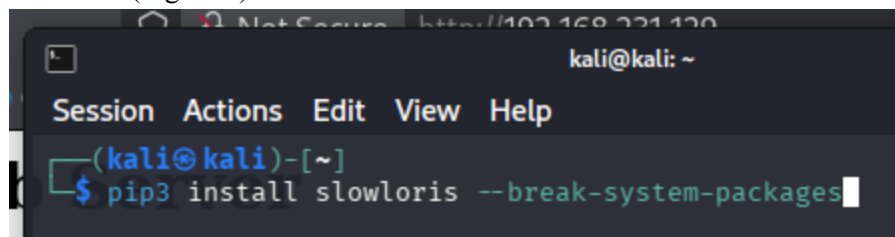


Figure 9. Slowloris installation

Technical Mechanism of the Slowloris Attack

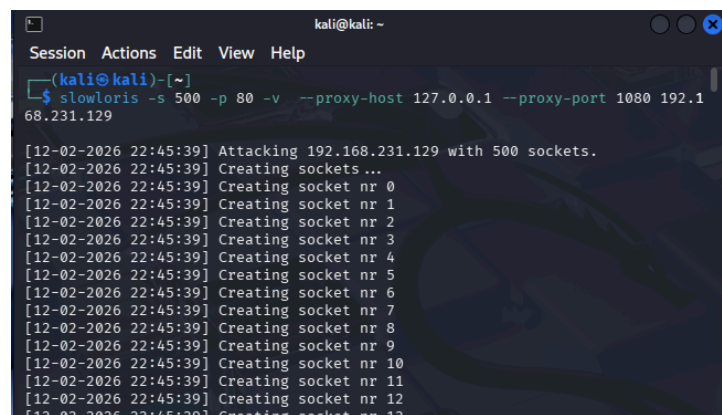
The Slowloris attack functions by exploiting the way thread-based web servers handle concurrent connections. In a standard HTTP interaction, a client sends a complete request, the server processes it and

sends a response, and then the connection is closed or returned to a pool [3]. Web servers have a finite number of slots available to handle these requests, often defined by the MaxRequestWorkers (previously MaxClients) directive.

The attack sequence follows these technical steps:

1. **Handshake and Initiation:** The attacker opens a TCP connection to the target server on port 80
2. **Partial Request Transmission:** The attacker sends an incomplete HTTP GET request. A complete request must terminate with two consecutive newlines (`\r\n\r\n`), but Slowloris only sends the first line and host header (e.g., `GET / HTTP/1.1\r\nHost: target.com\r\n`).
3. **Connection Hold:** The server, following the HTTP protocol standards, believes more data is coming and reserves a thread or process for this specific session, waiting for the request to finish [3].
4. **Header Trickling (Persistent State):** To prevent the connection from timing out due to inactivity, the attacker periodically sends a "dummy" header field (e.g., `X-a: b\r\n`) at intervals just shorter than the server's timeout (often every 10–20 seconds) [3].
5. **Slot Exhaustion:** The attacker repeats this process hundreds of times until all available MaxRequestWorkers slots are filled with idle, "partial" connections.

To start the attack, I use the following command on the Kali Linux VM (Figure 10):



```
kali@kali: ~  
Session Actions Edit View Help  
kali@kali ~  
$ slowloris -s 500 -p 80 -v --proxy-host 127.0.0.1 --proxy-port 1080 192.168.231.129  
[12-02-2026 22:45:39] Attacking 192.168.231.129 with 500 sockets.  
[12-02-2026 22:45:39] Creating sockets ...  
[12-02-2026 22:45:39] Creating socket nr 0  
[12-02-2026 22:45:39] Creating socket nr 1  
[12-02-2026 22:45:39] Creating socket nr 2  
[12-02-2026 22:45:39] Creating socket nr 3  
[12-02-2026 22:45:39] Creating socket nr 4  
[12-02-2026 22:45:39] Creating socket nr 5  
[12-02-2026 22:45:39] Creating socket nr 6  
[12-02-2026 22:45:39] Creating socket nr 7  
[12-02-2026 22:45:39] Creating socket nr 8  
[12-02-2026 22:45:39] Creating socket nr 9  
[12-02-2026 22:45:39] Creating socket nr 10  
[12-02-2026 22:45:39] Creating socket nr 11  
[12-02-2026 22:45:39] Creating socket nr 12  
[12-02-2026 22:45:39] Creating socket nr 13
```

Figure 10. Attack starting

- **slowloris** runs Slowloris DoS attack.
- **-s 500 -p 80 -v** creates 500 slow connections on port 80 (HTTP) and shows detailed output.
- **--proxy-host 127.0.0.1 --proxy-port 1080** routes the entire attack through a local SOCKS proxy to hide the attacker's IP.

This command performs a low-bandwidth Slowloris attack against 192.168.231.129:80 using 500 connections, with verbose logging, all hidden behind a local proxy.

To observe the effect of the attack, on the Red Hat VM, I use these command:

- **sudo tail -f /var/log/httpd/error_log**

This command shows the most recent Apache web server error messages and continuously updates the screen as new errors are written to the log.

- **while true; do**
 curl -s -m 30 -o /dev/null -w "%{http_code} %{time_total}s\n" http://192.168.231.129/
 sleep 3
done

This script continuously checks the availability of the web server at 192.168.231.129, printing the HTTP status code and how long each request takes, every 3 seconds.

The mathematical impact on service availability can be represented by the following equation (Figure 11):

$$A_{workers} = W_{max} - C_{attacker}$$

Figure 11. Equation to calculate the impact

Where $A_{workers}$ is the available capacity for legitimate users, W_{max} is the maximum workers allowed by the server, and $C_{attacker}$ is the number of malicious connections maintained by Slowloris. When $C_{attacker} \geq W_{max}$, the server enters a DoS state where $A_{workers} \leq 0$.

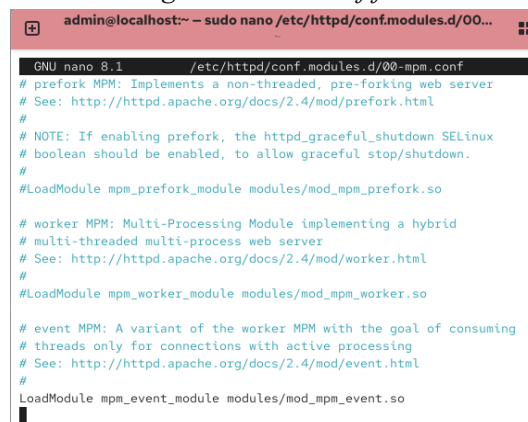
Implementation of Defensive Configurations

The primary architectural defense is the transition to the `mpm_event` module. In the older `mpm_prefork` model, each child process handles only one request at a time. If that request is a Slowloris connection, the entire process is tied up for the duration of the attack [11]. In contrast, `mpm_event` uses a hybrid multi-threaded approach with a dedicated "Listener" thread. When a connection is idle (waiting for data), the worker thread offloads the socket to the listener, freeing itself to serve other requests. If data arrives or a timeout occurs, the listener returns the socket to an available worker [12].

To setup `mpm_event` module, add the module to the sever config file using these command (Figure 12,13):

```
admin@localhost:~$ sudo nano /etc/httpd/conf.modules.d/00-mpm.conf  
[sudo] password for admin:
```

Figure 12. edit conf file



```
admin@localhost:~$ sudo nano /etc/httpd/conf.modules.d/00-mpm.conf  
GNU nano 8.1 /etc/httpd/conf.modules.d/00-mpm.conf  
# prefork MPM: Implements a non-threaded, pre-forking web server  
# See: http://httpd.apache.org/docs/2.4/mod/prefork.html  
#  
# NOTE: If enabling prefork, the httpd_graceful_shutdown SELinux  
# boolean should be enabled, to allow graceful stop/shutdown.  
#  
#LoadModule mpm_prefork_module modules/mod_mpm_prefork.so  
  
# worker MPM: Multi-Processing Module implementing a hybrid  
# multi-threaded multi-process web server  
# See: http://httpd.apache.org/docs/2.4/mod/worker.html  
#  
#LoadModule mpm_worker_module modules/mod_mpm_worker.so  
  
# event MPM: A variant of the worker MPM with the goal of consuming  
# threads only for connections with active processing  
# See: http://httpd.apache.org/docs/2.4/mod/event.html  
#  
#LoadModule mpm_event_module modules/mod_mpm_event.so
```

Figure 13. enable `mpm_event` module

The second layer of defense is the implementation of the `mod_reqtimeout` module (Figure 14), which is the official Apache solution for mitigating slow requests. The following sample configuration is applied to the target server:

```
GNU nano 8.1 /etc/httpd/conf.modules.d/00-base.conf
LoadModule mime_magic_module modules/mod_mime_magic.so
LoadModule mime_module modules/mod_mime.so
LoadModule negotiation_module modules/mod_negotiation.so
LoadModule remoteip_module modules/mod_remoteip.so
LoadModule reqtimeout_module modules/mod_reqtimeout.so
LoadModule request_module modules/mod_request.so
```

Figure 14. *mod_reqtimeout module*

```
admin@localhost:~ - sudo nano /etc/httpd/conf.d/reqtimeout....
GNU nano 8.1 /etc/httpd/conf.d/reqtimeout.conf
<IfModule mod_reqtimeout.c>
    RequestReadTimeout header=20-40,MinRate=500 body=20,MinRate=500
</IfModule>
```

Figure 15. *mod_reqtimeout module config*

In this configuration (Figure 15), the behavior is strictly enforced:

- **header=20-40:** The server waits an initial 20 seconds for the header. If the client sends data at the MinRate, the timeout extends up to 40 seconds.
- **MinRate=500:** The client must transmit data at a minimum of 500 bytes per second. Slowloris, which trickles only a few bytes (e.g., X-a: b\r\n), will fail this requirement, and the server will respond with a 408 Request Timeout error and drop the connection.

Analysis of Results

Phase 1: Unmitigated Baseline Observation

During the testing phase with default settings (mpm_prefork and no timeout modules), the Slowloris attack achieves a successful denial of service within seconds. The following symptoms are observed:

- **Service Failure:** Legitimate users attempting to browse the site experience an indefinite "connecting" state in their web browser. As you can see, when running the script (Figure 16) to check the availability of the web server at 192.168.231.129, all requests time out after 30 seconds, and curl return 000, which means that it did not get any HTTP responses. And when the client tried to access the webpage on Firefox, it took forever to load.

```
(kali@kali)-[~]
$ while true; do
  curl -s -m 30 -o /dev/null -w "%{http_code} %{time_total}s\n" http://192.1
68.231.129/
  sleep 3
done
000 30.002966s
000 30.002907s
000 30.002324s
000 30.002102s
000 30.002401s
000 30.002612s
000 30.002851s
000 30.002569s
000 30.002183s
```

Figure 16. *Server response time and status*

- **Log Entry AH00161:** The Apache error log records the following critical event (Figure 17):

```
[Wed Feb 11 13:28:39.749445 2026] [mpm_event:error] [pid 1365:tid 1365] AH10159:
server is within MinSpareThreads of MaxRequestWorkers, consider raising the Max
RequestWorkers setting
[Wed Feb 11 15:28:07.999794 2026] [mpm_prefork:error] [pid 11355:tid 11355] AH00
161: server reached MaxRequestWorkers setting, consider raising the MaxRequestWo
rkers setting
```

Figure 17. Apache error log

- **Connection Tracking:** Using “`netstat -antp | grep ':80|:443' | grep 192.168.231.128`” (Figure 18) and `watch -n 3 "netstat -ant | grep ':80|:443' | grep ESTABLISHED | grep 192.168.231.128 | wc -l"` (Figure 19), the analysis reveals hundreds of connections from the attacker’s IP address in an ESTABLISHED state, with zero data being transferred.

tcp6	337	0	192.168.231.129:80	192.168.231.128:58308	ESTABLISHED
tcp6	0	0	192.168.231.129:80	192.168.231.128:57330	ESTABLISHED
tcp6	0	0	192.168.231.129:80	192.168.231.128:57726	ESTABLISHED
tcp6	0	0	192.168.231.129:80	192.168.231.128:57376	ESTABLISHED
tcp6	0	0	192.168.231.129:80	192.168.231.128:56770	ESTABLISHED
tcp6	0	0	192.168.231.129:80	192.168.231.128:56760	ESTABLISHED
tcp6	338	0	192.168.231.129:80	192.168.231.128:59782	ESTABLISHED
tcp6	0	0	192.168.231.129:80	192.168.231.128:57900	ESTABLISHED
tcp6	0	0	192.168.231.129:80	192.168.231.128:57620	ESTABLISHED
tcp6	0	0	192.168.231.129:80	192.168.231.128:57778	ESTABLISHED
tcp6	0	0	192.168.231.129:80	192.168.231.128:57230	ESTABLISHED
tcp6	337	0	192.168.231.129:80	192.168.231.128:58404	ESTABLISHED

Figure 18. `netstat -antp | grep ':80|:443' | grep 192.168.231.128`

```
admin@localhost:~ -- watch -n 3 netstat -ant | grep ':80|:443' |...
Every 3.0s: netstat -ant | g... localhost.localdomain: Fri Feb 13 14:30:06 2026
500
```

Figure 19. `watch -n 3 "netstat -ant | grep ':80|:443' | grep ESTABLISHED | grep 192.168.231.128 | wc -l"`

- **Resource Paradox:** Despite the service failure, the server’s CPU usage remains remarkably low (< 2%), confirming that the bottleneck is not computational throughput but the exhaustion of the connection slot table (Figure 20).

```
admin@localhost:~ -- watch -n 2 ps -eo pid,%cpu,%mem,cmd -C...
Every 2.0s: ps -eo pid,%cpu,... localhost.localdomain: Fri Feb 13 14:30:30 2026

  PID %CPU %MEM    CMD
 41573  0.5  0.6  /usr/sbin/httpd -DFOREGROUND
 42902  0.4  0.3  systemd-userwork: waiting...
 2860  0.2  7.1  /usr/bin/gnome-shell
 1183  0.2  0.4  /usr/bin/vmtoolsd
 42894  0.2  0.3  systemd-userwork: waiting...
 3088  0.2  1.3  /usr/bin/vmtoolsd -n vmusr --uinputFd 3
 42954  0.1  0.3  /usr/sbin/httpd -DFOREGROUND
 42913  0.1  0.3  /usr/sbin/httpd -DFOREGROUND
 42834  0.1  0.3  systemd-userwork: waiting...
 3696  0.0 10.4  /usr/bin/ptxaxis --gaplication-service
 43040  0.0  0.3  /usr/sbin/httpd -DFOREGROUND
 43039  0.0  0.3  /usr/sbin/httpd -DFOREGROUND
 43037  0.0  0.3  /usr/sbin/httpd -DFOREGROUND
 42997  0.0  0.3  /usr/sbin/httpd -DFOREGROUND
```

Figure 20. CPU usage

Phase 2: Mitigated Result Observation

Upon implementing `mpm_event` and `mod_reqtimeout`, the server's resilience improves dramatically. While the attack is still active, the following results are recorded:

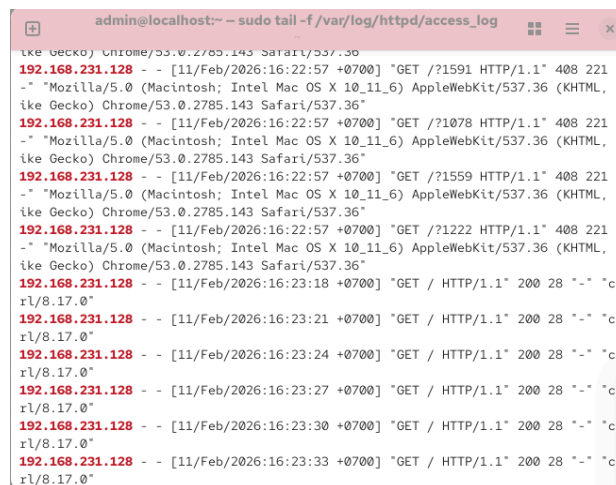
- **Continuous Availability:** Legitimate users can access the index page with nominal latency (< 1.5 seconds) despite the ongoing attack. As you can see, curl now returns status 200, and the response time (Figure 21) reduce **99.995030%** compare to the Phase 1 response time (Figure 16)



```
(kali@kali)-[~]
$ while true; do
  curl -s -m 30 -o /dev/null -w "%{http_code} %{time_total}s\n" http://192.1
68.231.129/
  sleep 3
done
200 13.659263s
200 0.003489s
200 0.001905s
200 0.001713s
200 0.003894s
200 0.002005s
200 0.001882s
200 0.002836s
200 0.002060s
200 0.002604s
200 0.001789s
200 0.001491s
200 0.001985s
```

Figure 21. Server response time and status

- **Log Entry 408:** The Apache access logs (Figure 22) fill with 408 Request Timeout status codes. This indicates that the mitigation is working correctly: identifying the slow connections and terminating them before they can exhaust the worker pool.



```
admin@localhost:~$ sudo tail -f /var/log/httpd/access_log
192.168.231.128 - - [11/Feb/2026:16:22:57 +0700] "GET /?1591 HTTP/1.1" 408 221 "
Mozilla/5.0 (Macintosh; Intel Mac OS X 10_11_6) AppleWebKit/537.36 (KHTML, l
ike Gecko) Chrome/53.0.2785.143 Safari/537.36"
192.168.231.128 - - [11/Feb/2026:16:22:57 +0700] "GET /?1078 HTTP/1.1" 408 221 "
Mozilla/5.0 (Macintosh; Intel Mac OS X 10_11_6) AppleWebKit/537.36 (KHTML, l
ike Gecko) Chrome/53.0.2785.143 Safari/537.36"
192.168.231.128 - - [11/Feb/2026:16:22:57 +0700] "GET /?1559 HTTP/1.1" 408 221 "
Mozilla/5.0 (Macintosh; Intel Mac OS X 10_11_6) AppleWebKit/537.36 (KHTML, l
ike Gecko) Chrome/53.0.2785.143 Safari/537.36"
192.168.231.128 - - [11/Feb/2026:16:22:57 +0700] "GET /?1222 HTTP/1.1" 408 221 "
Mozilla/5.0 (Macintosh; Intel Mac OS X 10_11_6) AppleWebKit/537.36 (KHTML, l
ike Gecko) Chrome/53.0.2785.143 Safari/537.36"
192.168.231.128 - - [11/Feb/2026:16:23:18 +0700] "GET / HTTP/1.1" 200 28 "-" cu
rl/8.17.0"
192.168.231.128 - - [11/Feb/2026:16:23:21 +0700] "GET / HTTP/1.1" 200 28 "-" cu
rl/8.17.0"
192.168.231.128 - - [11/Feb/2026:16:23:24 +0700] "GET / HTTP/1.1" 200 28 "-" cu
rl/8.17.0"
192.168.231.128 - - [11/Feb/2026:16:23:27 +0700] "GET / HTTP/1.1" 200 28 "-" cu
rl/8.17.0"
192.168.231.128 - - [11/Feb/2026:16:23:30 +0700] "GET / HTTP/1.1" 200 28 "-" cu
rl/8.17.0"
192.168.231.128 - - [11/Feb/2026:16:23:33 +0700] "GET / HTTP/1.1" 200 28 "-" cu
rl/8.17.0"
```

Figure 22. Apache error logs

- **watch -n 3 "apachectl status"** command reveals that the Apache server maintains a high total thread count (~841) while consistently showing only a small fraction of worker threads actively processing requests, which 72 idle workers and 27 busy workers out of the tracked pool, meaning roughly 27% busy (and ~73% idle at that moment) (Figure 23). This low active percentage - despite hundreds of incoming slow connections from the Slowloris attack - demonstrates that the single listener thread is efficiently managing a large number of sockets in wait-states and keep-alive pools. Most threads remain idle or parked with minimal overhead, preventing widespread worker exhaustion and keeping CPU usage very low, which is a hallmark of the event MPM's resilience against this type of low-bandwidth, connection-oriented denial-of-service attack.


```

Every 3.0s: apachectl status      localhost.localdomain: Fri Feb 13 15:33:30 2026

● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Fri 2026-02-13 15:29:46 +07; 3min 43s ago
  Invocation: 26f835b594b0445eb9ebdef8f0b661c3
     Docs: man:httpd.service(8)
    Main PID: 68142 (httpd)
   Status: "Total requests: 2242; Idle/Busy workers 72/27;Requests/sec: 10.3;
 Bytes served/sec: 6.3KB/sec"
      Tasks: 841 (limit: 10128)
     Memory: 85.4M (peak: 88.3M)
        CPU: 2.390s

```

Figure 23. Apache status

Comparative Metrics Analysis

The following table summarizes the performance data collected during both phases of the experiment:

Metric	Phase 1 (Unmitigated)	Phase 2 (Mitigated)
Legitimate User Access	Failed (Connection Timeout).	Success (Nominal Latency).
Apache Error Log	AH00161 (MaxWorkers Reached).	408 (Request Timeout).
Active Connections	100% of Worker Pool.	< 30% of Worker Pool.
System CPU Load	< 2% (Under-utilized).	3-5% (Nominal).

Table 7. Metrics analysis

The results confirm that Slowloris is a "semantic" attack rather than a "volumetric" one. It "wins" in the baseline scenario not by overpowering the server’s hardware, but by exploiting its willingness to wait for data that never arrives.

Evaluation

Attacker vs. Defender Perspectives

Figure 24 illustrates the MITRE ATT&CK Navigator matrix with the combination of three layers: Testing Scenario (score 1), Iran 2009 (score 2), and Internet Archive 2024 (score 3). In the matrix, light yellow represents a score of 1, orange represents a score of 2, light red represents a score of 3, and dark red represents a score of 6, indicating techniques common to all three attack scenarios.

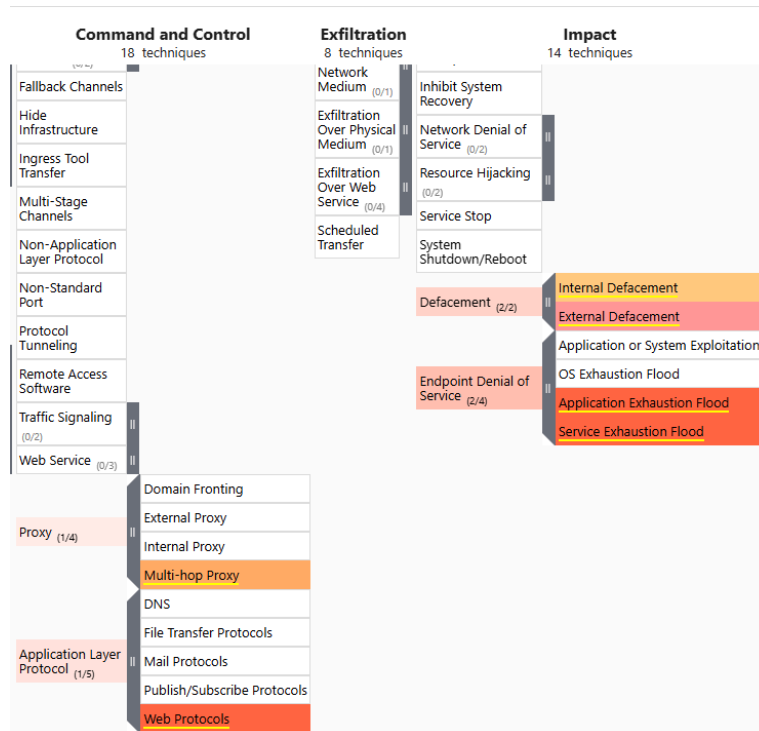


Figure 24. MITRE ATT&CK Navigator combination

Based on historical Slowloris attacks, with notable examples such as the Iran 2009 political protests and the Internet Archive 2024 incident, we can observe common characteristics among these application-layer denial-of-service attacks. Attackers consistently utilize legitimate HTTP protocols (T1071.001) to conduct low-bandwidth DoS attacks that exploit web server connection handling mechanisms. The attacks specifically target two critical sub-techniques: Service Exhaustion Flood (T1499.002) and Application Exhaustion Flood (T1499.003), which appear across all three scenarios (score 6, shown in dark red). The laboratory simulations employed Slowloris as the attack tool to execute partial HTTP header attacks against Apache 2.4 running `mpm_prefork`, while `mod_reqtimeout` and `mpm_event` were utilized on the defense side to detect and terminate slow connections. Despite the effectiveness of these mitigations in the controlled testing environment, several challenges emerge in real-world scenarios:

- Legitimate slow clients vs. malicious connections:** `mod_reqtimeout` enforces strict timeout policies that may inadvertently affect legitimate users on poor network connections (mobile users, satellite internet, congested networks). The `MinRate` parameter of 500 bytes per second, while effective against Slowloris, could cause false positives for users experiencing temporary network degradation.
- Distributed attack complexity:** As demonstrated in the Internet Archive 2024 attack, adversaries can leverage botnet infrastructure (T1583.006) and multi-hop proxies (T1090.003) to distribute the attack across thousands of IP addresses. This makes simple IP-based blocking insufficient, as each connection may appear to originate from a different legitimate source, requiring more sophisticated behavioral analysis.
- Resource overhead of connection management:** Transitioning to `mpm_event` improves resilience by offloading idle connections to a listener thread, but this architectural change requires careful configuration of `ThreadsPerChild`, `ServerLimit`, and `MaxRequestWorkers` parameters.

Misconfiguration can result in either insufficient capacity (leaving the server vulnerable) or excessive resource allocation (wasting system memory and CPU cycles during normal operation).

Essential Eight Mitigation Alignment

The ACSC Essential Eight provides a prioritized set of mitigation strategies to increase an organization’s cyber resilience [13]. While the framework is primarily designed to protect against malware and unauthorized access, several strategies directly support service availability and DoS protection.

Essential 8 Strategy	Relevance to Slowloris and DoS Resilience
Patch Applications	Organizations at all maturity levels must apply security updates for internet-facing services within 48 hours if an exploit exists [14]. This includes updating Apache to versions 2.2.15+ which include mod_reqtimeout by default.
User Application Hardening	Involves configuring applications to disable unnecessary features and enforce strict security baselines [15]. In this scenario, hardening includes setting aggressive timeouts and minimum data rates for HTTP interactions.
Patch Operating Systems	Updating the Linux kernel to support modern socket handling APIs (e.g., epoll), which are required for the efficient operation of mpm_event.
Restrict Admin Privileges	Ensuring that only authorized staff can modify web server configurations (e.g., changing MPMs or timeout values), reducing the risk of accidental misconfiguration that could lead to DoS vulnerability [15].

Table 8. Essential Eight Mitigation Alignment

Maturity Level 1 organizations are expected to establish a baseline of these controls, while higher maturity levels require more automated scanning and faster remediation timelines [16]. For DoS protection, alignment with the Essential Eight ensures that the underlying infrastructure is resilient, patched, and hardened against both known and emerging threats.

Future Outlook: Machine Learning and Automated Detection

While manual configuration of timeouts is effective, the future of DoS defense lies in the integration of Artificial Intelligence (AI) and Machine Learning (ML) for real-time anomaly detection. Research from 2022 and 2023 has demonstrated the potential of supervised and unsupervised learning models to identify

the faint traffic anomalies peculiar to Slowloris attacks [18].

Models such as Random Forest, Convolutional Neural Networks (CNN), and Gradient Boosting Machines have achieved over 95% accuracy in distinguishing between a legitimate slow client (e.g., a user on a poor mobile connection) and a malicious Slowloris adversary [19]. These systems can automatically adjust server thresholds or trigger IP-based blocking at the firewall level without the need for manual intervention by security teams. This adaptive defense represents the next phase of the "arms race" between attackers and defenders in the Layer 7 domain.

Conclusion

The Slowloris attack remains a potent threat to organizations that rely on legacy web server architectures and default configurations. Its ability to achieve service denial with minimal bandwidth makes it an ideal tool for hackers, state actors, and extortionists alike. This research has demonstrated that the vulnerability is rooted in the "wait-state" management of thread-based server models like Apache's `mpm_prefork`.

Through rigorous testing, it is concluded that architectural migration to event-driven models (`mpm_event`) combined with protocol-level enforcement (`mod_reqtimeout`) provides a highly effective and native defense against slow HTTP exploits. These mitigations align closely with the principles of the ACSC Essential Eight, particularly regarding application hardening and the timely patching of internet-facing services. To ensure continuous availability in an increasingly adversarial digital landscape, organizations must adopt a proactive security posture that accounts for semantic protocol exploits, transitioning from passive monitoring to automated, intelligent response systems.

References

- [1] "What is a denial of service attack (DoS) ?," *Paloaltonetworks.com*, 2019.
<https://www.paloaltonetworks.com/cyberpedia/what-is-a-denial-of-service-attack-dos> (accessed Feb. 12, 2026).
- [2] B. Menneni, "A Study of Slow DoS Attacks and Application Layer Defense," 2025.
Accessed: Feb. 12, 2026. [Online]. Available:
<https://dr.lib.iastate.edu/server/api/core/bitstreams/d305103c-d7eb-4a20-bb24-2cc7de9701b5/content>
- [3] "What is Slowloris?," *Learning Center*, 2023.
<https://www.imperva.com/learn/ddos/slowloris/> (accessed Feb. 12, 2026).
- [4] T. A. Nidecki, "Slowloris DDoS Attack," *Invicti.com*, 2025.
<https://www.invicti.com/learn/slowloris-attack> (accessed Feb. 12, 2026).
- [5] "Slowloris Denial of Service Attack," *Vercara*, Oct. 20, 2025.
<https://vercara.digicert.com/resources/slowloris-denial-of-service-attack> (accessed Feb. 12, 2026).
- [6] V. Sundar, "What is Slowloris," *Indusface*, Mar. 20, 2019.
<https://www.indusface.com/blog/what-is-slowloris/> (accessed Feb. 12, 2026).
- [7] Cloudflare, "What is the high orbit ion cannon (HOIC)?," *Cloudflare.com*, 2024.
<https://www.cloudflare.com/learning/ddos/ddos-attack-tools/high-orbit-ion-cannon-hoic/> (accessed Feb. 13, 2026).
- [8] jseidl, "jseidl/GoldenEye," *GitHub*, Dec. 26, 2019. <https://github.com/jseidl/GoldenEye> (accessed Feb. 12, 2026).

- [9] gkbrk, “gkbrk/slowloris,” *GitHub*, May 04, 2019. <https://github.com/gkbrk/slowloris> (accessed Feb. 12, 2026).
- [10] “How to Mitigate Slowloris Attacks,” *cPanel & WHM Documentation*, 2024. <https://docs.cpanel.net/knowledge-base/security/how-to-mitigate-slowloris-attacks/> (accessed Feb. 12, 2026).
- [11] “Apache MPMs – Prefork, Worker, and Event,” *Blog | Cloud Server & Hosting News | VPS.NET*, Apr. 08, 2013. <https://www.vps.net/blog/apache-mpms-prefork-worker-and-event/> (accessed Feb. 13, 2026).
- [12] D. Yordanov, “What are the differences between MPM Event and MPM Prefork in Plesk?,” *Plesk*, Jul. 09, 2024. <https://support.plesk.com/hc/en-us/articles/12388429705239-What-are-the-differences-between-MPM-Event-and-MPM-Prefork-in-Plesk> (accessed Feb. 13, 2026).
- [13] L. Friend, “Essential 8 Cyberthreat Mitigation Strategies,” *ConnectWise*, 2025. <https://www.connectwise.com/blog/acsc-essential-8-risk-mitigation-strategies-explained> (accessed Feb. 13, 2026).
- [14] Nexon, “Adopt the Essential 8 (ASD/ASC) as part of your cyber security strategy,” *Nexon Asia Pacific*, Sep. 25, 2021. <https://nexon.com.au/blog/adopt-the-essential-8-asd-asc-as-part-of-your-cyber-security-strategy/> (accessed Feb. 13, 2026).
- [15] A. Hinge, “ACSC Essential 8 Cybersecurity Strategies, Maturity Levels, and Best Practices,” *REAL Security*, Mar. 28, 2023. <https://www.real-sec.com/acsc-essential-8-cybersecurity-strategies-maturity-levels-and-best-practices/> (accessed Feb. 13, 2026).
- [16] A. Fitzgerald, “Essential Eight: A Breakdown of the Mitigation Strategies + Compliance Checklist,” *Secureframe*, Dec. 19, 2024. <https://secureframe.com/blog/essential-eight> (accessed Feb. 13, 2026).
- [17] “Endpoint Denial of Service: Service Exhaustion Flood, Sub-technique T1499.002 - Enterprise | MITRE ATT&CK®,” *attack.mitre.org*, Feb. 20, 2020. <https://attack.mitre.org/techniques/T1499/002/> (accessed Feb. 13, 2026).
- [18] Y. Swe, P. Pye Aung, and A. Su, “A Slow DDoS Attack Detection Mechanism using Feature Weighing and Ranking,” 2021. Accessed: Feb. 13, 2026. [Online]. Available: <https://www.ieomsociety.org/singapore2021/papers/797.pdf>
- [19] C. Jha and C. S. Dash, “Real-Time Slowloris Attack Detection and Mitigation with Machine Learning Techniques,” *International Journal of Engineering Research & Technology*, vol. 13, no. 9, Sep. 2024, doi: <https://doi.org/10.17577/IJERTV13IS090018>.